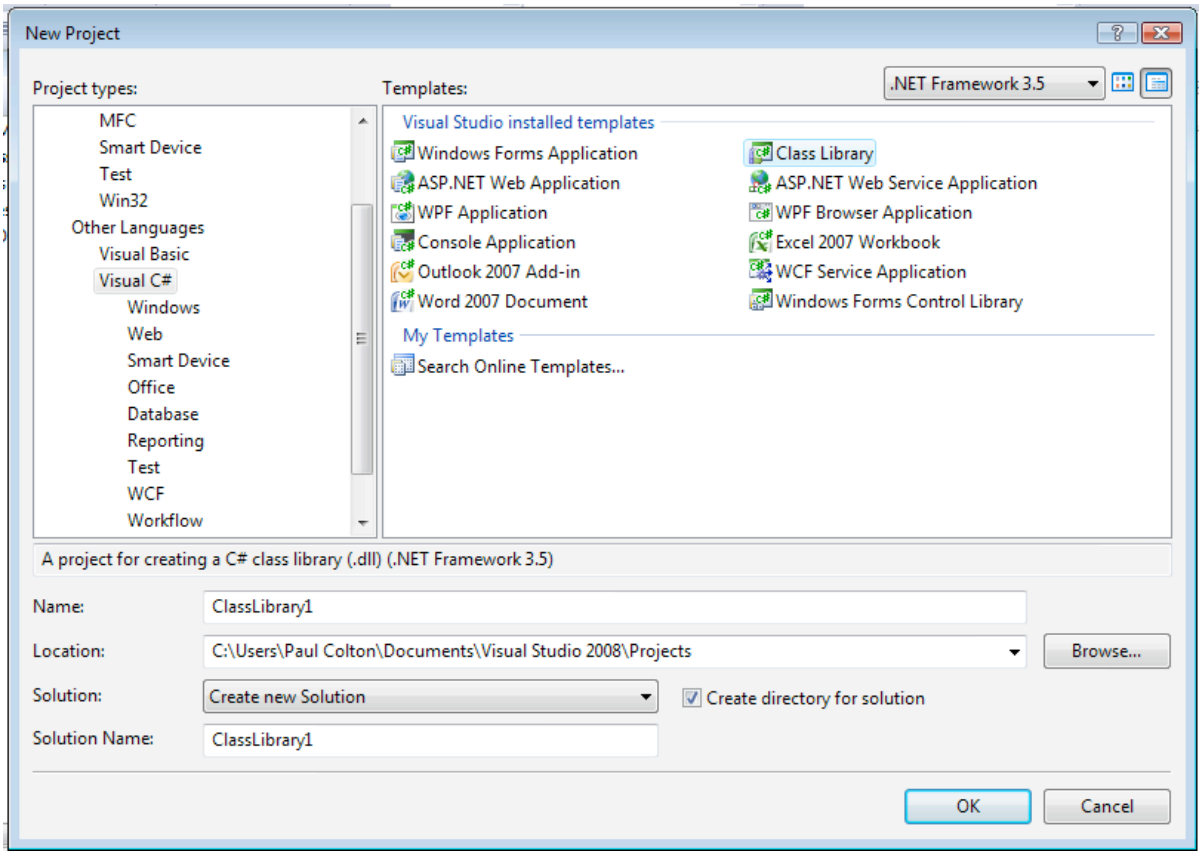
Jaxer and .NET: Image Rotation Example using C# and JS

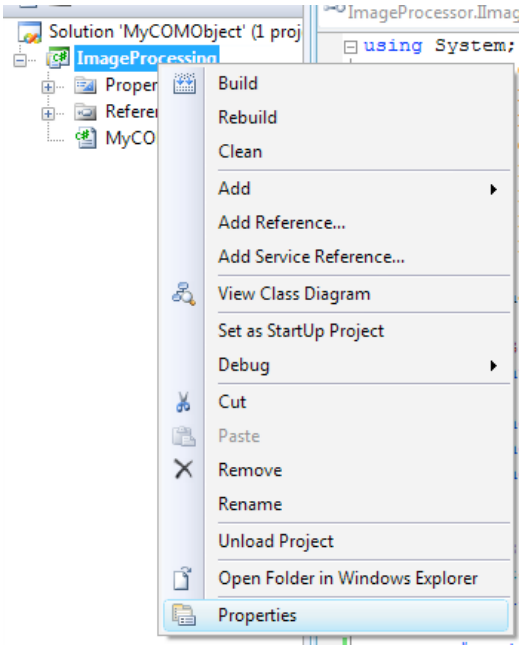
One of our users asked about extending Jaxer to do some image processing. He wanted to extend Jaxer using a language he had been already using for some of his back-end work, in this case, Microsoft's C# language which is part of .NET.

I quickly launched my VMWare Fusion into Vista, gulp, and proceeded to run Microsoft's Visual Studio 2008. This posting won't be a tutorial on how to write C#, but i'll just go over some of the basics of setting up a simple C#-based COM object that can be used from within Jaxer.

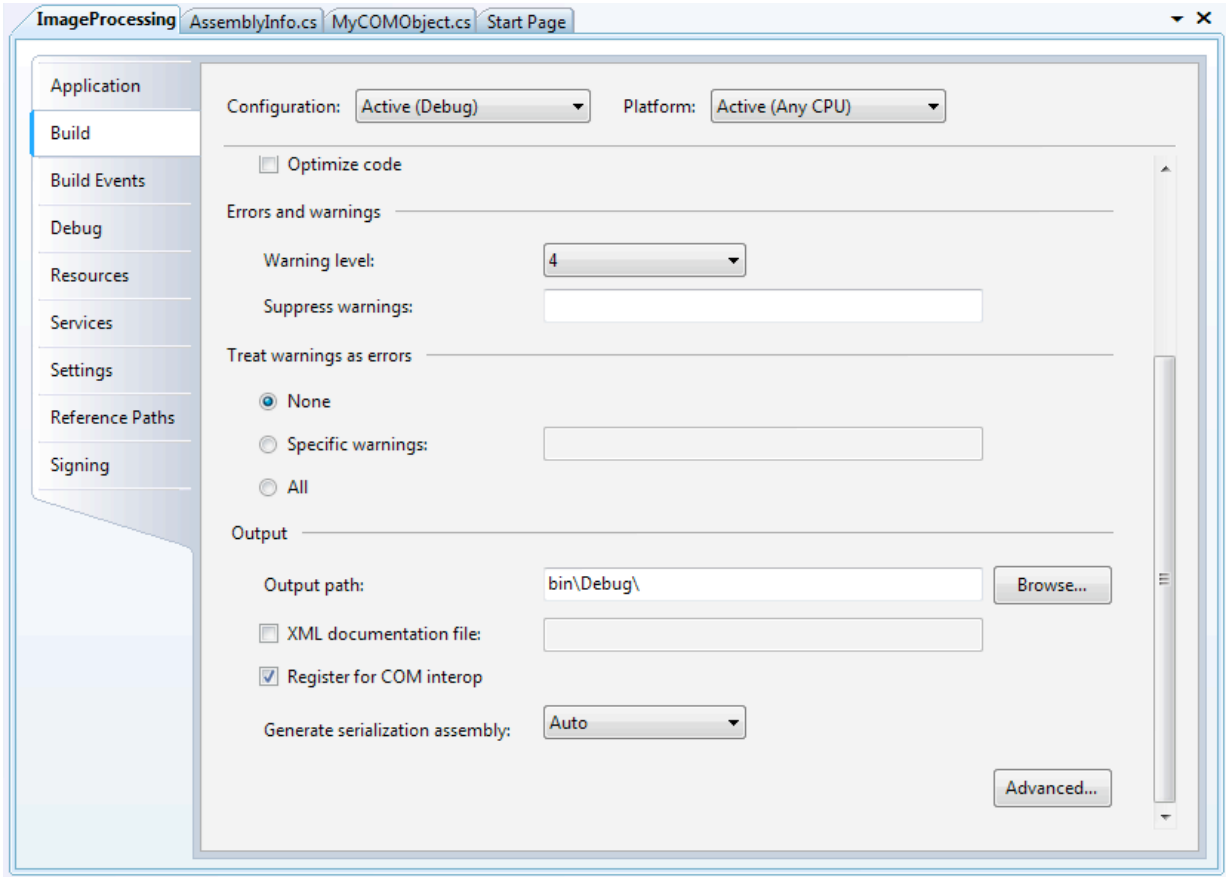
The first thing to note is that to register COM objects in Vista, you need to be Administrator, so just launch Visual Studio as Administrator using the right-click menu 'Run as Administrator'. Once you have Visual Studio running, start by creating a new 'Class Library' project.



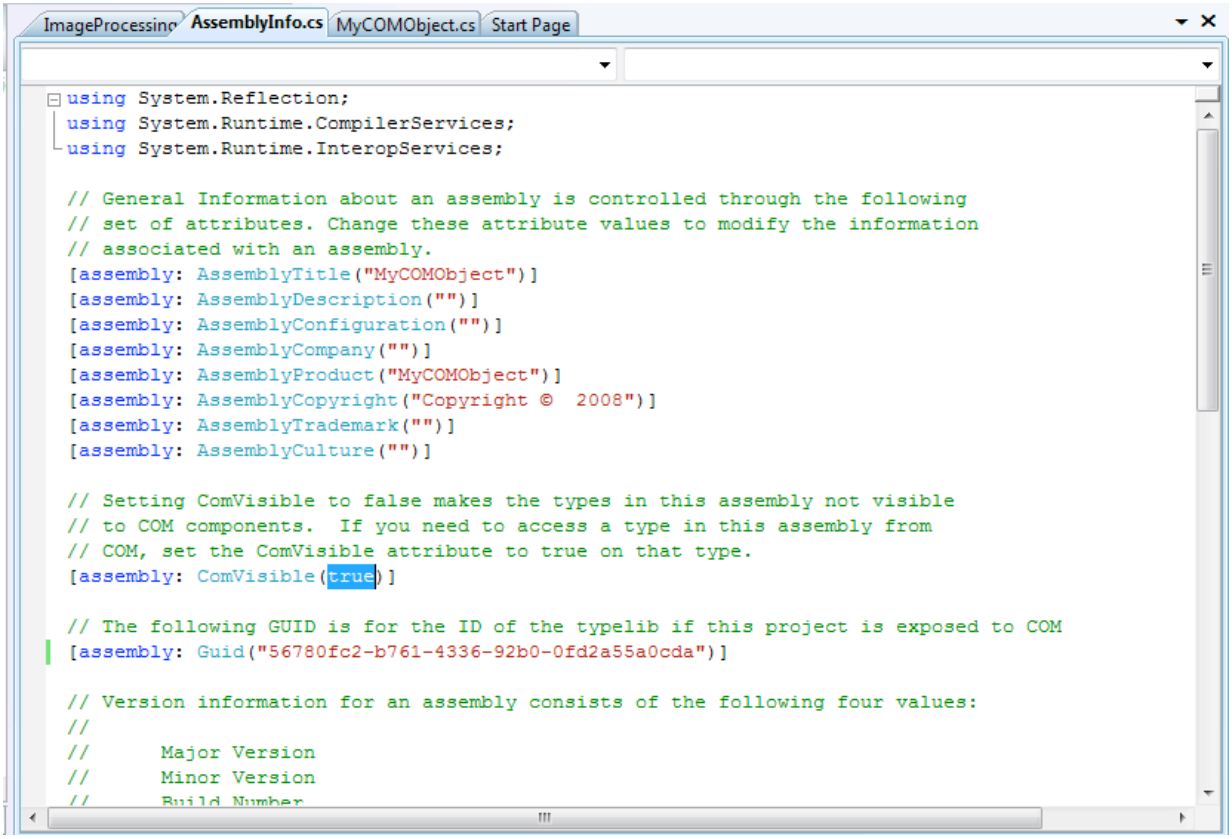
Once you've created the project, right-click on the project and go to the Properties menu, we'll want to make this object a COM-visible one.



In the 'Build' section of the properties, scroll to the bottom and check the 'Register for COM Interop' checkbox. This will make your object visible to other languages as a COM object.



The next thing you'll need to do is open your 'AssemblyInfo.cs' file, which is usually in your Properties folder of your project. Be sure to change the 'false' value to 'true' for the 'ComVisible' setting:



Now that you've done that, you can just create a COM object the 'usual' way. The complete project is attached, so you can take a look there. The image processing code was borrowed from the following online tutorial: http://blog.paranoidferret.com/index.php/2007/06/13/csharp-tutorial-image-editing-rotate/

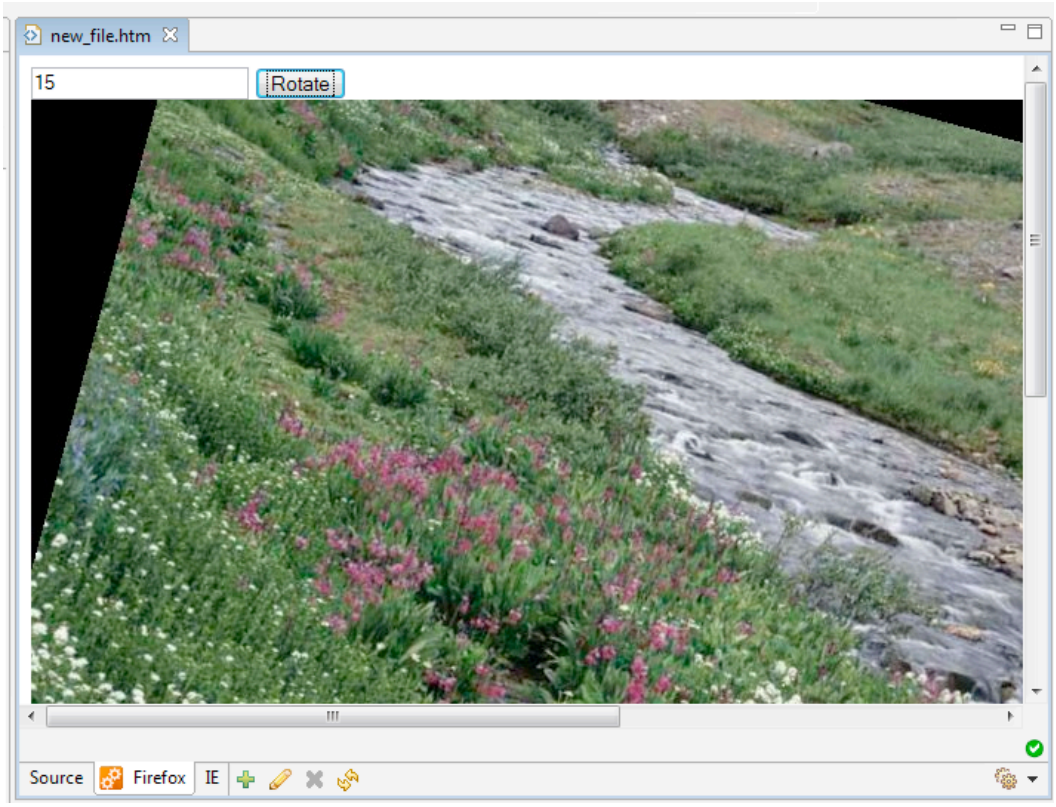
```
namespace ImageProcessor
{
    [Guid("BB49F18F-8B5A-4420-AF09-8C9AE3643297")]
    public interface IImageProcessing
    {
        string LoadImage(string filename);
        string RotateImage(float degrees);
        string SaveImage(string filename);
    }

    [Guid("589A999E-2AD4-466e-A9C8-E7D45D9D62E2")]
    [ClassInterface(ClassInterfaceType.None)]
    public class ImageProcessing : IImageProcessing
    {
        #region IImageProcessing Members

        public string LoadImage(string filename)
        {
```

Once you've got your object compiled, using it from within Jaxer is super-easy! You can just use the COMObject function to create a wrapped instance of the native object and then just use functions and properties on it just as if it were native JavaScript.

So here is the very short HTML file and JavaScript code running server-side via Jaxer that creates the COM object, loads our image, rotates it via the input field in the HTML form, and then saves it back as a new image. The communications from JS to .NET happens very quickly and as you can see, was super-simple to set up.

```
01. <html>
02.     <body>
03.         <script runat="server-proxy">
04.
05.             function rotate(angle)
06.             {
07.                 var img = COMObject("ImageProcessor.ImageProcessing");
08.
09.                 img.LoadImage(Jaxer.request.documentRoot + "/photo.jpg");
10.                 img.RotateImage(angle);
11.                 img.SaveImage(Jaxer.request.documentRoot + "/new.jpg");
12.             }
13.
14.         </script>
15.         <input id="angle">
16.         <input type="button" value="Rotate" onclick="rotate(document.getElementById('angle').value);
17.                                 document.getElementById('img').src =
18.                                 document.getElementById('img').src + '?' + new Date()">
19.         <br/>
20.         <img id="img" src="http://jaxer.org/tutorials/new.jpg">
21.     </body>
22. </html>
```

Here's a screenshot of the image being rotated:



What's really cool is that you can write anything you like in C#/VB.NET and easily access it via JavaScript in Jaxer. Note how in my sample, the onclick event for the button calls the server-side JavaScript function directly, which in turn calls the .NET COM object. That is, you can bind browser-side buttons directly to back-end .NET COM objects, all in JavaScript through Jaxer!